

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: ITA 0603

COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO5: *Introduction – NumPy Arrays – NumPy Basics- Math – Random – Indexing – Filtering – Statistics – Aggregation – saving data – Data analysis with Pandas – DataFrame – Combining – Indexing – File I/O – Grouping – Filtering – Sorting – Plotting (BL 3)*

Assessment Tool Weightage: 40%

QUESTIONS: 15

Total Marks:25

Annexure A

Constraint Based Problem Solving

Code of Conduct:

I Mohammed Mudhassir A/192424367(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Mudhassir

1. Constraint-Based Data Cleaning for Patient Records

A data cleaning pipeline can be designed using **Pandas** for handling the CSV data and **NumPy** for efficient numerical operations.

- Load the dataset using `pd.read_csv()`.
- Identify missing values using `df.isnull().sum()`.
- Fill numerical missing values with the **median** and categorical missing values with the **mode**, or remove records only when necessary.
- Use Boolean indexing and filtering to check medically valid ranges. For example, invalid ages, negative blood pressure values, or impossible heart rates can be removed or replaced with appropriate values.
- Standardize categorical values using methods such as `str.lower()`, `str.strip()`, and `replace()`.
- Use aggregation functions such as `mean()`, `median()`, `min()`, and `max()` to verify the cleaned data.

Example:

```
df["Age"] = df["Age"].fillna(df["Age"].median())
df["Gender"] = df["Gender"].fillna(df["Gender"].mode()[0])

df = df[(df["Age"] >= 0) & (df["Age"] <= 120)]
df["Gender"] = df["Gender"].str.strip().str.upper()

print(df.isnull().sum())
```

Thus, indexing and filtering enforce constraints while aggregation helps validate the final dataset and preserve maximum valid information.

2. Memory-Efficient Processing of a Massive NumPy Dataset

For a memory-limited system, NumPy arrays should use appropriate data types such as `float32` instead of `float64` when high precision is unnecessary.

- Create arrays efficiently using `np.arange()` or `np.empty()`.
- Process only required portions using slicing.
- Use vectorized operations instead of Python loops.
- Use in-place operations such as `arr *= 2`.
- Randomly sample only a small subset instead of copying the complete dataset.

- Use aggregation functions such as `np.mean()` and `np.sum()` directly.

Example:

```
import numpy as np
```

```
data = np.arange(1000000, dtype=np.float32)
```

```
sample = data[::100]
```

```
data *= 2
```

```
mean_value = np.mean(data)
```

For debugging, check `data.nbytes`, monitor execution time, and inspect array shape and dtype. If memory overflow occurs, reduce precision, process data in chunks, or use memory-mapped arrays.

3. Combining Customer Transaction and Demographic DataFrames

The two DataFrames can be combined using `pd.merge()` based on `Customer_ID`.

First, duplicate IDs should be identified using:

```
df[df["Customer_ID"].duplicated()]
```

Then duplicates can be removed or grouped depending on whether multiple transactions are valid. A one-to-one demographic table can be validated before merging.

```
customers = customers.drop_duplicates(subset="Customer_ID")
```

```
merged = pd.merge(
    transactions,
    customers,
    on="Customer_ID",
    how="inner"
)
```

```
merged = merged.sort_values("Customer_ID").reset_index(drop=True)
```

Filtering can identify missing or unmatched records, while `groupby("Customer_ID")` can validate duplicate counts. An outer merge with the merge indicator can be used to debug mismatched IDs:

```
check = pd.merge(
    transactions,
    customers,
    on="Customer_ID",
    how="outer",
    indicator=True
)
```

Records marked as `left_only` or `right_only` indicate missing matches.

4. Efficient Statistical Analysis with NumPy and Pandas

Statistical measures can be calculated using built-in aggregation functions:

```
mean = df["Value"].mean()
median = df["Value"].median()
variance = df["Value"].var()
```

With NumPy:

```
import numpy as np
```

```
mean = np.mean(data)
median = np.median(data)
variance = np.var(data)
```

To improve efficiency, irrelevant or invalid values should first be filtered:

```
valid_data = data[data >= 0]
```

Numerical accuracy can be improved by using suitable data types such as `float64` when necessary. Redundant calculations should be avoided by storing frequently used results.

For debugging, check:

- Data type using `dtype`
 - Missing values
 - Array/DataFrame shape
 - Unexpected outliers
 - Differences between NumPy and Pandas results
-

5. Complete Data Analysis Pipeline

A complete pipeline contains the following stages:

1. Loading

Use efficient functions such as:

```
df = pd.read_csv("data.csv")
```

For large files, load selected columns or use chunksize.

2. Processing

Clean missing values, standardize column names, and convert data types.

```
df.columns = df.columns.str.strip().str.lower()
```

3. Filtering

Apply user-defined conditions using Boolean indexing:

```
filtered = df[df["sales"] > 1000]
```

4. Aggregation and Sorting

```
result = df.groupby("category")["sales"].sum()
```

```
result = result.sort_values(ascending=False)
```

5. Visualization

```
import matplotlib.pyplot as plt
```

```
result.plot(kind="bar")
```

```
plt.show()
```

For debugging, handle corrupted files using exception handling, inspect column names and data types, and validate the data before plotting.

6. Cleaning Financial Transaction Data

The pipeline should first remove duplicate records:

```
df = df.drop_duplicates()
```

Timestamps can be standardized using:

```
df["Timestamp"] = pd.to_datetime(  
    df["Timestamp"],  
    errors="coerce"  
)
```

Invalid transactions, such as negative amounts or missing IDs, can be filtered:

```
df = df[df["Amount"] > 0]
```

```
df = df.dropna(subset=["Transaction_ID", "Timestamp"])
```

Sorting ensures chronological consistency:

```
df = df.sort_values("Timestamp")
```

Grouping by transaction or customer ID can help detect unusual duplicate patterns:

```
df.groupby("Customer_ID")["Amount"].sum()
```

Indexing, grouping, and sorting therefore help maintain data integrity while preserving valid records.

7. Real-Time Sensor Data Processing

For low-latency processing, sensor values should be stored in NumPy arrays and processed using vectorized operations.

Filtering

```
valid = data[(data >= min_value) & (data <= max_value)]
```

Normalization

```
normalized = (valid - valid.min()) / (  
    valid.max() - valid.min()  
)
```

Anomaly Detection

```
mean = np.mean(valid)
```

```
std = np.std(valid)
```

```
anomalies = valid[  
    np.abs(valid - mean) > 3 * std  
]
```

Slicing can process only recent sensor readings:

```
recent = data[-1000:]
```

Vectorization avoids slow loops, while aggregation functions calculate statistics efficiently. Debugging includes checking processing time, array size, shapes, and threshold values.

8. Merging Multiple Product Datasets

First, ensure that all DataFrames have a consistent schema:

```
df.columns = df.columns.str.lower().str.strip()
```

Merge datasets using a common product identifier:

```
merged = pd.merge(  
    df1,  
    df2,  
    on="product_id",  
    how="outer"  
)
```

An outer merge helps avoid data loss. Conflicting values can be resolved by prioritizing a trusted source or using `combine_first()`.

Grouping can identify duplicate products:

```
merged.groupby("product_id").size()
```

Filtering can locate missing or conflicting attributes. Finally, validate the schema, remove unnecessary duplicates, and reset the index.

9. Preprocessing Millions of Mixed-Type Records

Efficient preprocessing begins by assigning appropriate data types:

```
df["Age"] = pd.to_numeric(  
    df["Age"],  
    errors="coerce"  
)
```

Categorical columns can use the category data type:

```
df["Department"] = df["Department"].astype("category")
```

Missing numerical values can be filled using median values:

```
df["Age"] = df["Age"].fillna(  
    df["Age"].median()  
)
```

Categorical missing values can be replaced with the mode or "Unknown".

NumPy vectorized operations should be used instead of loops. After processing:

```
df = df.reset_index(drop=True)
```

This ensures consistent indexing and improves efficiency for machine learning preprocessing.

10. Rolling Statistics on Time-Series Data

First, convert the time column into datetime format and sort it:

```
df["Date"] = pd.to_datetime(df["Date"])
```

```
df = df.sort_values("Date")
```

Set the time column as the index:

```
df = df.set_index("Date")
```

Calculate moving average and variance:

```
df["Moving_Average"] = df["Value"].rolling(  
    window=7  
) .mean()
```

```
df["Rolling_Variance"] = df["Value"].rolling(  
    window=7  
) .var()
```

Pandas rolling operations are optimized and avoid manually recalculating each window. Debugging should check whether the data is correctly sorted, whether the window size is appropriate, and whether missing values affect the output.

11. Combining Healthcare CSV and JSON Data

The CSV file can be loaded using:

```
csv_df = pd.read_csv("patients.csv")
```

JSON data can be loaded using:

```
json_df = pd.read_json("patients.json")
```

Nested JSON fields can be flattened using:

```
normalized = pd.json_normalize(json_data)
```

Both datasets should be converted into a common schema before merging. Column names and data types should be standardized.

```
final_df = pd.concat(  
    [normalized, csv_df]  
)
```



```
[csv_df, normalized],  
ignore_index=True  
)
```

Indexing helps align records, normalization handles nested JSON, and aggregation can validate record counts and missing values. Debugging includes comparing column names, checking data types, and identifying schema mismatches.

12. Real-Time Financial Risk Metrics

NumPy can efficiently calculate returns:

```
returns = np.diff(prices) / prices[:-1]
```

Volatility can be calculated using:

```
volatility = np.std(returns)
```

Correlation between assets can be calculated using:

```
correlation = np.corrcoef(asset_returns)
```

Slicing avoids unnecessary copying:

```
recent_prices = prices[-1000:]
```

Broadcasting allows mathematical operations across multiple assets without loops.

Aggregation functions such as `mean()` and `std()` provide fast risk metrics.

Debugging involves checking for zero or missing prices, verifying array shapes, monitoring execution time, and comparing calculated results with small test datasets.

13. Cleaning Social Media Post Data

Duplicate posts can be removed using:

```
df = df.drop_duplicates(  
    subset=["Post_Text"]  
)
```

Spam entries can be filtered using Boolean conditions or text patterns:

```
df = df[  
    ~df["Post_Text"].str.contains(  
        "spam",  
        case=False,  
        na=False
```

```
)  
]
```

Text can be normalized using:

```
df["Post_Text"] = (  
    df["Post_Text"]  
    .str.lower()  
    .str.strip()  
)
```

Timestamps should be standardized:

```
df["Timestamp"] = pd.to_datetime(  
    df["Timestamp"],  
    errors="coerce"  
)
```

Grouping by user can identify unusually frequent posting behavior. Debugging includes inspecting invalid timestamps, missing text, duplicate counts, and unexpected characters.

14. Feature Extraction from High-Dimensional Data

A scalable pipeline can use Pandas for data preparation and NumPy for efficient numerical computation.

Feature Selection

Remove constant or irrelevant features:

```
df = df.loc[  
    :,  
    df.nunique() > 1  
]
```

Correlation or variance analysis can identify redundant features.

Normalization

```
X = df.to_numpy(dtype=np.float32)
```

```
X = (  
    X - X.mean(axis=0)
```

```
) / X.std(axis=0)
```

Dimensionality Reduction

Techniques such as PCA can reduce the number of features while retaining important information.

Vectorization processes all features efficiently, while indexing selects only useful columns. Debugging includes checking feature shapes, zero standard deviation columns, missing values, and duplicated features.

15. Integrating IoT Data with Different Sampling Rates

First, convert timestamps to a common format:

```
df["Timestamp"] = pd.to_datetime(
    df["Timestamp"]
)
```

Set timestamps as the index:

```
df = df.set_index("Timestamp").sort_index()
```

Different devices can be aligned using `reindex()` or `resample()`:

```
df = df.resample("1min").mean()
```

Missing intervals can be handled using interpolation:

```
df = df.interpolate(method="time")
```

Data from multiple devices can then be merged using their timestamps. Aggregation functions such as `mean()`, `max()`, and `min()` summarize values over common time intervals.

Debugging should verify:

- Timestamps are sorted.
- Time zones are consistent.
- Sampling intervals are correct.
- Interpolation does not create unrealistic values.
- Merged DataFrames have expected record counts.

Conclusion

For all these problems, the key principles are:

- **NumPy** provides fast array processing, vectorization, slicing, broadcasting, and numerical aggregation.

- **Pandas** provides powerful tools for loading, cleaning, filtering, grouping, merging, sorting, indexing, and time-series analysis.
- Efficient **filtering and indexing** reduce unnecessary computation.
- **Aggregation functions** help summarize and validate data.
- Proper debugging using shape, dtype, isnull(), duplicated(), descriptive statistics, and small test samples ensures correctness and data integrity.